# Problem A. 排列

## 算法 1：

直接枚举排列，判断排列是否可爱。

复杂度：$O(n*(n!))$，期望得分：20 分。

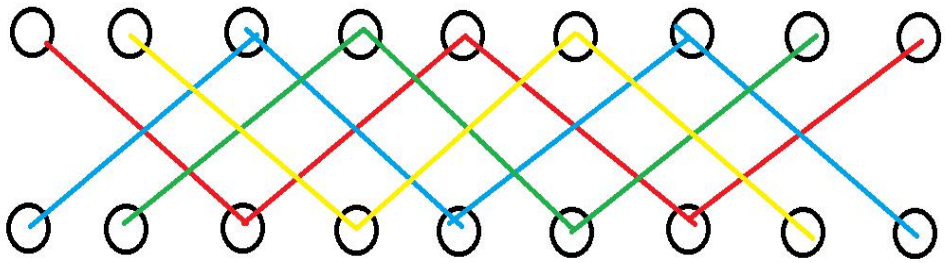
## 算法 2：

状态压缩动态规划，f[S]表示填好了 S 集合里的数且都满足要求的方案数。枚举下一位在哪填即可转移。

复杂度：$O(n*(2^n))$，期望得分：40 分。

## 算法 3：

模型转换，我们发现这是个二分图，上 i 相连下 j 表示 pi 不能在 j 的位置，那么图长这个样子：



我们发现这个图就是由一堆不相交的链组成，而我们要求的就是这个二分图补图的最大匹配的数量。

考虑容斥，令 g[i]为选 i 条这个二分图上的边的方案数。那么答案就是：

$$ans = \sum_{i=0}^{n} g[i]*(n-i)!*(-1)^i$$

现在要做的就是如何求 g[i]，我们发现每条链可以分开做。

首先链上相邻的边不能选，因为一个点只能匹配另一边的一个点。

所以 f[i][j][0/1]表示做到链上的第 i 个点，选了 j 条边，第 i 号点有没有选上。

那么很明显：

1. f[i][j][0] = (f[i - 1][j][0] + f[i - 1][j][1]) （不选）
2. f[i][j][1] = f[i - 1][j - 1][0] （选）

最后就可以枚举每条链上选多少条边来算出 g 了。

做一个 f 的复杂度是链长的平方，就是$O((n/k)^2)$，然后做了 2k 条链，最后对所有链合并。

所以总的复杂度是：$O(n^2/k+n^2)$也就是$O(n^2)$，期望得分：95 分。

## 算法 4：

这 5 分算是有些超纲了吧。

基于算法 3 的想法，我们考虑如何优化。

我们发现 f[i][j][1]是多余的，因为它等于 f[i-1][j-1][0]。

那么我们去掉 0/1，式子变成 f[i][j]=f[i-1][j]+f[i-2][j-1]。

通过观察我们可以得知 f[i][j]=C(i-j,j)。（C 是组合数，证明用归纳法即可）

我们最后需要的只是 f[len][i](i<=len)，len 是链长，而链长只有两种不同的！

所以把它们提出来，我们记两种链长，一个是 len，一个肯定是 len+1，它们出现次数为 cnt1 和 cnt2，因为合并是卷积，那么我们把卷积的式子写出来就是：

$$g = f[len]^{cnt_1} * f[len+1]^{cnt_2}$$

我们发现这个卷积完大小肯定不会超过 2n（二分图中边的个数）。

所以直接用大小为 2n 的 NTT 算法将它们卷起来即可。

因为出题人素质比较高，做人比较良心，所以没有将模数定为 1e9+7，当然，如果你开心，你也可以写 MTT。

复杂度：$O(n \log n)$，期望得分：100 分。

# Problem B. 苹果树

## 算法 1：

枚举删哪一条边，接着用 tarjan 算法求出所有割边，如果是非树边就会有 1 的贡献，否则没有。

复杂度：$O(n^2)$，期望得分：60 分。

## 算法 2：

我们发现如果一条树边有贡献则树上最多只有一条新加的边覆盖了它，那么我们只要写一下 LCA（最近公共祖先算法）然后树上差分一下，被新边覆盖了恰好一次的树边就会有 1 的贡献，而没有被覆盖的有 m 的贡献，直接统计一下就好了。

复杂度：$O(n \log n)$ 或 $O(n)$（取决于 LCA 算法的复杂度），期望得分：90~100 分。

# 括号序列

先考虑判断一个串是否存在一个合法的转化方案（下面简称合法）。可以用一个栈维护当前还没有被匹配的字符，从左往右扫描字符串，如果当前字符和栈顶元素相同，则弹栈，否则把当前字符压入栈中。这个串合法的充要条件是最后栈为空。

这样可以在 $O(n)$ 的时间内判断以 $i$ 开头的所有字符串中有多少个是合法的，总的判断时间复杂度为 $O(n^2)$.

现在考虑把复杂度降下来。我们不从每一个位置 $i$ 开始扫字符串，就只扫描整个字符串一遍。这样我们得到了 $n+1$ 个栈 $s_0$ 至 $s_n$。一个结论是区间 $[l,r]$ 是合法的当前仅当 $s_{l-1}=s_r$。

利用这个结论，我们可以求出 $s_0$ 至 $s_n$ 的哈希值（因为单次的操作只有弹栈和插入一个元素，所以可以很方便地维护 Hash 值），然后排序求其中值相等的无序对数即可。时间复杂度为 $O(n\log n)$。

关于上面的结论，可以对 $s_{l-1}$ 和 $s_r$ 的最长公共前缀长度归纳。当最长公共前缀长度为 $0$ 的时候显然成立。

现在考虑消去 $s_{l-1}$ 和 $s_r$ 的第一位元素。如果他们不同，假设分别是 $a$ 和 $b$，则这个串一定形如 $S_1aS_2aS_3bS$，其中 $S_1,S_2,S_3$ 均为合法串，$l-1$ 的位置在 $S_1$ 中，$r$ 的位置在 $S$ 中。此时如果从 $l$ 开始用栈扫描串的话，这个 $b$ 一定消除不掉。

假设他们不同，如果第一位元素在原串中的位置相同，显然这一位可以忽略，如果位置不同，则串形如 $S_1aS_2aS_3aS$，同样 $S_1,S_2,S_3$ 均为合法串，$l-1$ 的位置在 $S_1$ 中，$r$ 的位置在 $S$ 中。因此从 $l$ 开始扫的话两个 $a$ 会互相匹配掉，因此只需要考虑第二个 $a$ 后的部分，这是最长公共前缀长度减一的情况。

因此归纳成立。